

Big picture: what you are building

Final system:

```
None
C code on ARM processor
    |
    | AXI-Lite control registers
    v
Custom matrix accelerator in FPGA
    ^
    | AXI DMA streams data from/to DDR
    v
DDR memory
```

Your job is to make this work step by step:

```
None
Software matrix multiply
→ Verilog MAC simulation
→ Zybo Hello World
→ AXI-Lite register test
→ tiny PL matrix multiply without DMA
→ DMA loopback
→ DMA pass-through accelerator
→ real matrix multiply accelerator
→ 8 PEs
→ tiling
```

Do **not** start with 64x64, 8 PEs, tiling, and double buffering. That is the final version.

Phase 0: Understand the parts

Before coding, know these names:

Term	Meaning
PS	Processing System, the ARM Cortex-A9 on the Zybo
PL	Programmable Logic, the FPGA fabric
DDR	External RAM shared by ARM and FPGA
AXI-Lite	Simple register bus, used for control
AXI DMA	Xilinx IP that moves data between DDR and AXI-Stream
MM2S	Memory-mapped to stream: DDR → accelerator
S2MM	Stream to memory-mapped: accelerator → DDR
AXI-Stream	Data pipe with <code>tdata</code> , <code>tvalid</code> , <code>tready</code> , <code>tlast</code>
BRAM	Small on-chip FPGA memory
PE	Processing Element, one multiply-accumulate unit
FSM	Finite State Machine, hardware controller

Phase 1: Start fully in software

Do this first on your Mac.

Goal

Prove you understand matrix multiplication and the data format.

Use:

```
None
A: int8
B: int8
C: int32
```

Start with:

None

`M = 4`

`K = 4`

`N = 4`

So:

None

`A is 4x4`

`B is 4x4`

`C is 4x4`

Folder structure

Create this:

None

`matrix_accel_project/`

`software_model/`

`main.c`

`rtl/`

`tb/`

`vitis/`

`vivado/`

Code first: `software_model/main.c`

Write normal C code:

C/C++

`#include <stdio.h>`

`#include <stdint.h>`

`#define M 4`

`#define K 4`

```

#define N 4

int main() {
    int8_t A[M][K] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {1, 1, 1, 1},
        {-1, 2, -3, 4}
    };

    int8_t B[K][N] = {
        {1, 0, 2, 1},
        {0, 1, 2, 1},
        {1, 1, 0, 1},
        {2, 0, 1, 1}
    };

    int32_t C[M][N] = {0};

    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            int32_t sum = 0;
            for (int k = 0; k < K; k++) {
                sum += (int32_t)A[i][k] * (int32_t)B[k][j];
            }
            C[i][j] = sum;
        }
    }

    printf("C =\n");
    for (int i = 0; i < M; i++) {
        for (int j = 0; j < N; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }
}

```

```
    return 0;
}
```

Execute on Mac

```
Shell
cd matrix_accel_project/software_model
gcc main.c -o matmul
./matmul
```

Pass condition

You see a correct 4x4 result printed.

Commit this:

```
Shell
git init
git add .
git commit -m "software matrix multiply reference"
```

This software result becomes your **golden reference**. Later, hardware must match this.

Phase 2: Write the smallest hardware block: one PE

A PE does this:

```
None
acc = acc + a * b
```

where:

None

```
a = signed int8
b = signed int8
acc = signed int32
```

File: rtl/pe.v

None

```
module pe (
    input wire      clk,
    input wire      rst_n,
    input wire      clear,
    input wire      enable,
    input wire signed [7:0] a,
    input wire signed [7:0] b,
    output reg  signed [31:0] acc
);

always @(posedge clk) begin
    if (!rst_n) begin
        acc <= 32'sd0;
    end else if (clear) begin
        acc <= 32'sd0;
    end else if (enable) begin
        acc <= acc + (a * b);
    end
end

endmodule
```

File: tb/tb_pe.v

None

```
`timescale 1ns/1ps
```

```
module tb_pe;

reg clk;
reg rst_n;
reg clear;
reg enable;
reg signed [7:0] a;
reg signed [7:0] b;
wire signed [31:0] acc;

pe dut (
    .clk(clk),
    .rst_n(rst_n),
    .clear(clear),
    .enable(enable),
    .a(a),
    .b(b),
    .acc(acc)
);

always #5 clk = ~clk;

initial begin
    clk = 0;
    rst_n = 0;
    clear = 0;
    enable = 0;
    a = 0;
    b = 0;

    #20;
    rst_n = 1;

    clear = 1;
    #10;
```

```

clear = 0;

enable = 1;

a = 8'sd2;
b = 8'sd3;
#10; // acc = 6

a = -8'sd4;
b = 8'sd5;
#10; // acc = 6 - 20 = -14

enable = 0;
#10;

if (acc == -14)
    $display("PASS: acc = %0d", acc);
else
    $display("FAIL: acc = %0d", acc);

$finish;
end

endmodule

```

Execute on Mac

If you have Icarus Verilog:

```

Shell
cd matrix_accel_project
iverilog -g2012 -o tb/pe_test tb/tb_pe.v rtl/pe.v
vvp tb/pe_test

```

Pass condition

You should see:

None

PASS: acc = -14

Do not continue until this works.

Phase 3: Run Hello World on the Zybo

Now move to the lab computer with **Vivado/Vitis**.

Goal

Prove you can program the board and run ARM software.

In Vivado

1. Open Vivado.
2. Create a new project.
3. Choose an RTL project.
4. Choose your exact Zybo board:
 - Zybo Z7-10 or Zybo Z7-20, depending on your board.
5. Create Block Design.
6. Add IP:

None

ZYNQ7 Processing System

7.

Click:

None

Run Block Automation

8.

Make sure these are connected:

None

DDR

FIXED_IO

9.

Click:

None

Validate Design

10.

Create HDL Wrapper.

11. Generate Bitstream.

12. Export Hardware:

None

File → Export → Export Hardware

Select:

None

Include bitstream

13.

This creates an `.xsa` file.

In Vitis

1. Create new workspace.
2. Create platform project from the `.xsa`.
3. Create application project.
4. Choose:

None

```
hello_world
```

5. Build.
6. Connect Zybo USB cable.
7. Open serial terminal:

None

```
115200 baud
```

8. Program FPGA.
9. Run the application.

Pass condition

UART prints:

None

```
Hello World
```

Do not touch DMA yet.

Phase 4: Run software matrix multiply on ARM only

Now take your `main.c` matrix multiply and run it on the ARM processor.

Goal

Prove your C reference works on the Zybo PS.

What to code

In Vitis, replace Hello World with your 4x4 matrix multiply C code.

Use:

```
C/C++  
#include <stdio.h>  
#include <stdint.h>  
#include "xil_printf.h"
```

Use `xil_printf()` instead of normal `printf()` if needed.

Execute

1. Build application.
2. Program FPGA.
3. Run application.
4. Check UART output.

Pass condition

The ARM prints the same C matrix that your Mac printed.

At this point:

```
None  
Software reference works on Mac  
Software reference works on Zybo ARM
```

Good.

Phase 5: Create AXI-Lite register IP

Now you will make the ARM talk to custom FPGA logic.

Do **not** do matrix multiplication yet.

Goal

ARM writes registers in PL and reads them back.

Register map

Use this:

None

Offset	Name	Purpose
0x00	CTRL	bit 0 = start
0x04	STATUS	bit 0 = done, bit 1 = busy
0x08	DIM_M	
0x0C	DIM_K	
0x10	DIM_N	
0x14	RESULT	

In Vivado

1. Go to:

None

Tools → Create and Package New IP

- 2.

Choose:

None

Create a new AXI4 peripheral

- 3.

Name:

None

`matrix_accel`

4.

Interface type:

None

`AXI4-Lite Slave`

5.

Number of registers:

None

`6`

6.

Finish.

7. Edit IP.

Vivado creates an AXI-Lite wrapper. Inside it, there will be registers like:

None

`slv_reg0`

`slv_reg1`

`slv_reg2`

`slv_reg3`

`slv_reg4`

`slv_reg5`

Map them like this:

None

`slv_reg0 = CTRL`

`slv_reg1 = STATUS`

```
slv_reg2 = DIM_M  
slv_reg3 = DIM_K  
slv_reg4 = DIM_N  
slv_reg5 = RESULT
```

For now, make **STATUS** and **RESULT** simple.

Example behavior:

None

```
When CTRL start bit is written:  
    busy becomes 1 briefly  
    done becomes 1  
    result becomes M + K + N
```

So if software writes:

None

```
M = 4  
K = 4  
N = 4
```

then **RESULT** should become:

None

```
12
```

Connect in Vivado block design

Add your custom IP to the block design.

You should have:

None

```
ZYNQ7 Processing System
```

```
AXI Interconnect or SmartConnect
matrix_accel
Processor System Reset
```

Connect:

```
None
Zynq M_AXI_GP0 → AXI Interconnect S00_AXI
AXI Interconnect M00_AXI → matrix_accel S_AXI
FCLK_CLK0 → matrix_accel s_axi_aclk
peripheral_aresetn → matrix_accel s_axi_aresetn
```

Then:

1. Open Address Editor.
2. Assign address.
3. Validate design.
4. Generate bitstream.
5. Export hardware with bitstream.
6. Update Vitis platform.

In Vitis

Check `xparameters.h`.

Find something like:

```
C/C++
#define XPAR_MATRIX_ACCEL_0_S00_AXI_BASEADDR ...
```

Names may differ.

Write test code:

```
C/C++
#include "xil_io.h"
#include "xil_printf.h"
```



```

#include "xparameters.h"

#define ACCEL_BASE XPAR_MATRIX_ACCEL_0_S00_AXI_BASEADDR

#define REG_CTRL    0x00
#define REG_STATUS  0x04
#define REG_M       0x08
#define REG_K       0x0C
#define REG_N       0x10
#define REG_RESULT  0x14

int main() {
    xil_printf("AXI-Lite register test\r\n");

    Xil_Out32(ACCEL_BASE + REG_M, 4);
    Xil_Out32(ACCEL_BASE + REG_K, 4);
    Xil_Out32(ACCEL_BASE + REG_N, 4);

    Xil_Out32(ACCEL_BASE + REG_CTRL, 1);

    u32 status = Xil_In32(ACCEL_BASE + REG_STATUS);
    u32 result = Xil_In32(ACCEL_BASE + REG_RESULT);

    xil_printf("STATUS = %d\r\n", status);
    xil_printf("RESULT = %d\r\n", result);

    return 0;
}

```

Pass condition

UART prints:

None

RESULT = 12

Now you have proven:

None

ARM can control custom FPGA registers

This is a huge milestone.

Phase 6: Tiny PL matrix multiply without DMA

Now add real computation, but still **no DMA**.

Goal

Custom PL IP computes a hardcoded 4x4 matrix multiply.

The ARM only does:

None

write start

wait done

read result/checksum

Why hardcoded first?

Because DMA adds a lot of complexity. First prove the accelerator FSM works.

What to code in hardware

Inside `matrix_accel`, add:

None

A[4][4] hardcoded

B[4][4] hardcoded

```
C[4][4] internal
FSM with states:
  IDLE
  CLEAR
  COMPUTE
  DONE
```

You can compute one multiply per clock using your `pe.v`.

Pseudo-hardware:

```
None
on start:
  i = 0
  j = 0
  k = 0
  sum = 0

for each i:
  for each j:
    sum = 0
    for each k:
      sum += A[i][k] * B[k][j]
    C[i][j] = sum
```

At the end, compute a checksum:

```
None
RESULT = sum of all C values
STATUS.done = 1
```

What to connect

Nothing new in block design.

Still only:

None

PS → AXI-Lite → matrix_accel

What to test

In Vitis:

C/C++

```
Xil_Out32(ACCEL_BASE + REG_CTRL, 1);

while ((Xil_In32(ACCEL_BASE + REG_STATUS) & 1) == 0) {
    // wait
}

result = Xil_In32(ACCEL_BASE + REG_RESULT);
```

Pass condition

The hardware checksum equals the software-computed checksum.

Now you have:

None

A real FPGA computation controlled by ARM

Still no DMA. That is okay.

C/C++

Phase 7: Add AXI DMA loopback

Now test DMA separately.

Do **not** connect DMA to your matrix accelerator yet.

Goal

Send data from DDR through DMA and receive it back unchanged.

None

DDR tx_buffer → DMA MM2S → AXI-Stream loopback → DMA S2MM → DDR rx_buffer

In Vivado block design

Add:

None

AXI DMA

AXI4-Stream Data FIFO, optional but helpful
SmartConnect or AXI Interconnect

Configure AXI DMA:

None

Scatter Gather: disabled

MM2S channel: enabled

S2MM channel: enabled

Stream data width: 32 bits for beginner MVP

Your proposal uses 64-bit stream width, but 32-bit is easier for the first working version. You can change to 64-bit later.

Enable HP0 port

Double click Zynq PS.

Enable:

None

S_AXI_HP0

This lets DMA access DDR.

Connect control path

None

Zynq M_AXI_GP0 → AXI Interconnect

AXI Interconnect → AXI DMA S_AXI_LITE

AXI Interconnect → matrix_accel S_AXI

Connect DMA memory path

None

AXI DMA M_AXI_MM2S → SmartConnect → Zynq S_AXI_HP0

AXI DMA M_AXI_S2MM → SmartConnect → Zynq S_AXI_HP0

Connect stream loopback

For the loopback test:

None

AXI DMA M_AXIS_MM2S → AXI DMA S_AXIS_S2MM

or:

None

AXI DMA M_AXIS_MM2S → AXI4-Stream FIFO → AXI DMA S_AXIS_S2MM

Connect clocks

Use the same clock first:

None

```
Zynq FCLK_CLK0 → DMA clocks  
Zynq FCLK_CLK0 → interconnect clocks  
Zynq FCLK_CLK0 → matrix_accel clock
```

Connect resets

None

```
proc_sys_reset peripheral_aresetn → active-low reset inputs
```

Validate, bitstream, export

Same as before:

None

```
Validate Design  
Generate Bitstream  
Export Hardware with bitstream  
Update Vitis platform
```

Vitis DMA loopback code

Use Xilinx's AXI DMA simple polling example as your starting point.

Basic flow:

C/C++

```
XAxiDma dma;  
XAxiDma_Config *cfg;  
  
cfg = XAxiDma_LookupConfig(XPAR_AXIDMA_0_DEVICE_ID);  
XAxiDma_CfgInitialize(&dma, cfg);  
  
if (XAxiDma_HasSg(&dma)) {  
    xil_printf("DMA is in SG mode; expected simple mode\r\n");  
}
```

```
    return -1;
}
```

Create buffers:

```
C/C++
#define LEN 16

u32 tx_buffer[LEN] __attribute__((aligned(64)));
u32 rx_buffer[LEN] __attribute__((aligned(64)));
```

Fill:

```
C/C++
for (int i = 0; i < LEN; i++) {
    tx_buffer[i] = i + 100;
    rx_buffer[i] = 0;
}
```

Cache handling:

```
C/C++
Xil_DCacheFlushRange((UINTPTR)tx_buffer, LEN * sizeof(u32));
Xil_DCacheFlushRange((UINTPTR)rx_buffer, LEN * sizeof(u32));
```

Start receive first:

```
C/C++
XAxiDma_SimpleTransfer(
    &dma,
    (UINTPTR)rx_buffer,
    LEN * sizeof(u32),
    XAXIDMA_DEVICE_TO_DMA
);
```


Start send:

```
C/C++
XAxiDma_SimpleTransfer(
    &dma,
    (UINTPTR)tx_buffer,
    LEN * sizeof(u32),
    XAXIDMA_DMA_TO_DEVICE
);
```

Wait:

```
C/C++
while (XAxiDma_Busy(&dma, XAXIDMA_DMA_TO_DEVICE));
while (XAxiDma_Busy(&dma, XAXIDMA_DEVICE_TO_DMA));
```

Invalidate cache before reading result:

```
C/C++
Xil_DCacheInvalidateRange((UINTPTR)rx_buffer, LEN * sizeof(u32));
```

Compare:

```
C/C++
for (int i = 0; i < LEN; i++) {
    if (rx_buffer[i] != tx_buffer[i]) {
        xil_printf("FAIL at %d\r\n", i);
        return -1;
    }
}

xil_printf("DMA LOOPBACK PASS\r\n");
```

Pass condition

UART prints:

None

DMA LOOPBACK PASS

Do not continue until this works.

This proves:

None

DDR → DMA → stream → DMA → DDR works

Phase 8: Replace loopback with pass-through accelerator

Now your custom IP receives stream data and sends it back.

Still no matrix multiplication.

Goal

Test AXI-Stream handshaking.

None

DDR tx → DMA MM2S → your IP → DMA S2MM → DDR rx

What your accelerator does

For each input word:

None

output = input

or:

None

```
output = input + 1
```

Start with an exact copy.

What to add to custom IP

Add AXI-Stream input:

None

```
s_axis_tdata  
s_axis_tvalid  
s_axis_tready  
s_axis_tlast
```

Add AXI-Stream output:

None

```
m_axis_tdata  
m_axis_tvalid  
m_axis_tready  
m_axis_tlast
```

Connect in Vivado

Remove the stream loopback.

Connect:

None

```
AXI DMA M_AXIS_MM2S → matrix_accel S_AXIS  
matrix_accel M_AXIS → AXI DMA S_AXIS_S2MM
```

Keep AXI-Lite connected too:

None

```
PS M_AXI_GP0 → matrix_accel S_AXI
```

Test from Vitis

Use the same DMA loopback code.

Expected:

None

```
rx_buffer[i] == tx_buffer[i]
```

Pass condition

UART prints:

None

```
PASS-THROUGH ACCELERATOR PASS
```

This proves:

None

```
DMA can communicate with your custom hardware
```

This is another huge milestone.

Phase 9: First real matrix accelerator through DMA

Now implement actual matrix multiply.

Still keep it small.

Start with fixed dimensions

Use:

```
None  
M = 4  
K = 4  
N = 4
```

Do not use tiling yet.

Data format

Use one input buffer:

```
None  
input_buffer = [A bytes][B bytes]
```

For 4x4:

```
None  
A has 16 int8 values  
B has 16 int8 values  
Total input = 32 bytes
```

Output buffer:

```
None  
output_buffer = [C int32 values]
```

For 4x4:

```
None  
C has 16 int32 values  
Total output = 64 bytes
```

Memory layout

Row-major:

None

$A[i][k]$ stored at $A[i*K + k]$

$B[k][j]$ stored at $B[k*N + j]$

$C[i][j]$ stored at $C[i*N + j]$

Hardware states

Your accelerator FSM should look like:

None

IDLE

LOAD_A

LOAD_B

COMPUTE

SEND_C

DONE

IDLE

Wait for `CTRL.start`.

LOAD_A

Read $M*K$ int8 values from AXI-Stream and store into internal memory.

For 4x4:

None

read 16 bytes

LOAD_B

Read $K*N$ int8 values from AXI-Stream and store into internal memory.

For 4x4:

None

```
read 16 bytes
```

COMPUTE

Use nested counters:

None

```
i from 0 to M-1  
j from 0 to N-1  
k from 0 to K-1
```

Compute:

None

```
C[i*N + j] += A[i*K + k] * B[k*N + j]
```

SEND_C

Send each `C` value as one 32-bit AXI-Stream word.

For 4x4:

None

```
send 16 words
```

Set `tlast = 1` on the final word.

DONE

Set:

None

```
STATUS.done = 1  
STATUS.busy = 0
```

Important beginner simplification

Use 32-bit AXI-Stream.

Packing input int8 into 32-bit stream means each input word contains 4 matrix values:

None

```
word[7:0]    = byte 0
word[15:8]   = byte 1
word[23:16]  = byte 2
word[31:24]  = byte 3
```

So for 4x4:

None

```
A = 16 bytes = 4 stream words
B = 16 bytes = 4 stream words
Total input = 8 stream words
```

Output is easier:

None

```
Each C is int32 = one stream word
C = 16 stream words
```

Vitis send sequence

Use this order:

C/C++

```
// 1. Fill input buffer with A and B
// 2. Flush input buffer
// 3. Flush/invalidate output buffer
// 4. Configure M, K, N
// 5. Start S2MM receive
// 6. Start accelerator
// 7. Start MM2S send
```



```
// 8. Wait for DMA
// 9. Wait for accelerator done
// 10. Invalidate output buffer
// 11. Compare with software C
```

Example sequence:

```
C/C++
Xil_Out32(ACCEL_BASE + REG_M, 4);
Xil_Out32(ACCEL_BASE + REG_K, 4);
Xil_Out32(ACCEL_BASE + REG_N, 4);

Xil_DCacheFlushRange((UINTPTR)input_buffer, input_bytes);
Xil_DCacheFlushRange((UINTPTR)output_buffer, output_bytes);

XAxiDma_SimpleTransfer(
    &dma,
    (UINTPTR)output_buffer,
    output_bytes,
    XAXIDMA_DEVICE_TO_DMA
);

Xil_Out32(ACCEL_BASE + REG_CTRL, 1);

XAxiDma_SimpleTransfer(
    &dma,
    (UINTPTR)input_buffer,
    input_bytes,
    XAXIDMA_DMA_TO_DEVICE
);

while (XAxiDma_Busy(&dma, XAXIDMA_DMA_TO_DEVICE));
while (XAxiDma_Busy(&dma, XAXIDMA_DEVICE_TO_DMA));

while ((Xil_In32(ACCEL_BASE + REG_STATUS) & 1) == 0);
```

```
Xil_DCacheInvalidateRange((UINTPTR)output_buffer, output_bytes);
```

Pass condition

For every element:

```
C/C++  
C_hw[i] == C_sw[i]
```

UART prints:

```
None  
MATRIX MULTIPLY 4x4 PASS
```

This is your first real project demo.

Phase 10: Scale from 4x4 to 8x8, then 16x16

Do not jump to 64x64.

Use this order:

```
None  
4x4  
8x8  
16x16  
32x32  
64x64
```

At each size, test:

None

all positive values
mixed positive/negative values
zero matrix
identity matrix
random small values

Recommended value range

Use small int8 values first:

None

-5 to +5

This makes debugging easier.

Print only small matrices

For 4x4 and 8x8, print full matrices.

For 16x16 and larger, print:

None

PASS/FAIL
first mismatch index
expected value
actual value

Example:

C/C++

```
if (C_hw[idx] != C_sw[idx]) {  
    xil_printf("FAIL idx=%d expected=%d actual=%d\r\n",  
               idx, C_sw[idx], C_hw[idx]);  
}
```

Phase 11: Add 8 PEs

Only after single-PE matrix multiply works.

Goal

Compute 8 output columns in parallel.

For one row i , compute columns:

None

$j, j+1, j+2, \dots, j+7$

Use 8 accumulators:

None

$acc0, acc1, acc2, acc3, acc4, acc5, acc6, acc7$

For each k :

None

$a = A[i][k]$

$acc0 += a * B[k][j+0]$

$acc1 += a * B[k][j+1]$

$acc2 += a * B[k][j+2]$

$\dots$

$acc7 += a * B[k][j+7]$

This is much easier than a full systolic array.

New compute loop

Instead of:

None

```
for i
  for j
    for k
```

Use:

None

```
for i
  for j_block in steps of 8
    clear 8 accumulators
    for k
      broadcast A[i][k]
      feed B[k][j_block + 0..7]
    store 8 C values
```

Why this is good

For $N = 8$, you compute one whole row group at once.

For $N = 16$, you do two groups:

None

```
columns 0..7
columns 8..15
```

Pass condition

Same as before:

None

```
C_hw == C_sw
```

UART:

None

8-PE MATRIX MULTIPLY PASS

Now your design matches the spirit of the proposal's 8 MAC array.

Phase 12: Add larger matrix support

Now move toward the proposal's 64x64 maximum.

Without tiling

For 64x64:

None

A = 64*64 int8 = 4096 bytes

B = 64*64 int8 = 4096 bytes

C = 64*64 int32 = 16384 bytes

This is actually not huge.

For the first 64x64 version, you may store A, B, and C in BRAM/register memory if resources allow.

But the proposal's final design uses tile buffers because larger designs need better BRAM management.

Suggested order

None

4x4 single PE

8x8 single PE

8x8 8 PEs

16x16 8 PEs

32x32 8 PEs

64x64 8 PEs

Only after this should you think about tiling.

Phase 13: Add tiling only after everything works

Tiling is final-level complexity.

What tiling means

Instead of computing the whole matrix at once, you compute pieces.

Example tile size:

None
32x32

The proposal says initial tile size is 32x32 and uses ping-pong BRAM buffers.

But for implementation, do this:

Tiling version 1: no double buffering

Compute:

```
None
for tile_i
  for tile_j
    clear C_tile
    for tile_k
      load A_tile
      load B_tile
      compute partial sum
    write C_tile
```

Tiling version 2: double buffering

Only if time remains:

None

```
while computing current tile:  
    load next tile into second buffer
```

This is an optimization. It should not be your first goal.

Exact order you should follow

Use this checklist.

Checkpoint 1 — Software reference on Mac

Do:

None

```
write C matrix multiply  
run with gcc  
print C
```

Pass:

None

```
correct 4x4 output
```

Checkpoint 2 — PE simulation

Do:

None

```
write pe.v  
write tb_pe.v
```



```
simulate
```

Execute:

Shell

```
iverilog -g2012 -o tb/pe_test tb/tb_pe.v rtl/pe.v  
vvp tb/pe_test
```

Pass:

None

```
PASS: acc = -14
```

Checkpoint 3 — Zybo Hello World

Do:

None

```
Vivado Zynq block design  
export XSA  
Vitis hello_world  
run on board
```

Pass:

None

```
Hello World over UART
```

Checkpoint 4 — ARM software matrix multiply

Do:

None

```
run same C matrix multiply on Zybo ARM
```

Pass:

None

```
same result as Mac
```

Checkpoint 5 — AXI-Lite register IP

Connect:

None

```
PS M_AXI_GP0 → AXI Interconnect → matrix_accel S_AXI
```

Code:

None

```
CTRL, STATUS, M, K, N, RESULT registers
```

Test:

None

```
write M/K/N  
write start  
read result
```

Pass:

None

```
RESULT = M + K + N
```

Checkpoint 6 — Hardcoded PL matrix multiply

Connect:

```
None
same AXI-Lite only
```

Code:

```
None
hardcoded A/B inside FPGA
FSM computes checksum
```

Test:

```
None
start accelerator
wait done
read checksum
```

Pass:

```
None
hardware checksum == software checksum
```

Checkpoint 7 — DMA loopback

Connect:

```
None
PS HP0 enabled
DMA MM2S/S2MM memory ports → HP0
DMA AXI-Lite → GP0
DMA stream output → DMA stream input
```

Code:

None

```
tx_buffer  
rx_buffer  
XAxiDma_SimpleTransfer  
cache flush/invalidate
```

Pass:

None

```
rx_buffer == tx_buffer
```

Checkpoint 8 — Streaming pass-through accelerator

Connect:

None

```
DMA MM2S → matrix_accel S_AXIS  
matrix_accel M_AXIS → DMA S2MM
```

Code hardware:

None

```
copy input stream to output stream
```

Pass:

None

```
rx_buffer == tx_buffer
```

Checkpoint 9 — 4x4 matrix multiply through DMA

Connect:

None

same as checkpoint 8

Code hardware:

None

LOAD_A

LOAD_B

COMPUTE

SEND_C

Code software:

None

pack A and B into input buffer

start receive DMA

start accelerator

start send DMA

compare C_hw with C_sw

Pass:

None

MATRIX MULTIPLY 4x4 PASS

Checkpoint 10 — 8x8 and 16x16

Do:

None

increase internal memory

increase counters

test more cases

Pass:

None

`C_hw == C_sw`

Checkpoint 11 — 8 PEs

Code:

None

`8 parallel accumulators
broadcast A
feed 8 B values
compute 8 C columns at once`

Pass:

None

`8-PE output matches software`

Checkpoint 12 — 32x32 / 64x64

Do:

None

`scale dimensions
watch BRAM usage
watch timing`

Pass:

None

`64x64 PASS`

Checkpoint 13 — Tiling

Only if the previous checkpoints work.

Do:

None

```
32x32 tiles  
partial sums  
tile loop FSM
```

Pass:

None

```
tilled result matches software
```

Common bugs and what they mean

DMA hangs forever

Usually one of these:

None

```
tready is not asserted  
tvalid is not handled  
tlast is missing  
wrong transfer length  
S2MM was not started before output  
DMA not connected to HP0  
cache issue
```

Hardware returns all zeros

Usually:

None

accelerator never started
AXI-Lite base address wrong
input stream not received
cache not flushed
reset stuck active

First few values correct, rest wrong

Usually:

None

counter bug
wrong row-major indexing
wrong byte packing
missing signed conversion
wrong tlast position

Software sees old result

Usually:

None

forgot Xil_DCCacheInvalidateRange on output buffer

Hardware sees old input

Usually:

None

forgot Xil_DCCacheFlushRange on input buffer

What you should do today

Start with only these three things:

None

1. Create `software_model/main.c` and run 4x4 matrix multiply on your Mac.
2. Write `pe.v` and `tb_pe.v` and simulate one multiply-accumulate.
3. Open Vivado and get Hello World running on the Zybo.

That is the correct beginner starting point. Once those three work, move to AXI-Lite registers.